

Algorithms Notes — Skiena (TADM 3e) + CLRS (4e), with C++ from Weiss (4e)

Start at [INDEX.md](#) — module map, CLRS↔Skiena crosswalk, study order, and the C++ conventions used throughout.

What is in this archive

16 markdown files: `INDEX.md` plus `M01`–`M15`. Every module is a unified synthesis of Skiena and CLRS on one topic, with all implementation code in **C++17**.

The refinement pass — complete for all 15 modules

using 647 `std::` prefixes removed. Every block assumes the prelude `#include`
namespace `<bits/stdc++.h> + using namespace std;`
std;

everywhere

Every code block compiles 30 translation units (a body TU and an appendix TU per module) verified under `g++ -std=c++17 -Wall -Wextra`. Illustrative fragments were rewritten into real compilable functions rather than left as snippets.

All links resolve 435 links checked, **0 broken**. Fixed a genuinely broken filename (`M13-graph-traversal.md`) and repointed 17 forward-references to unwritten modules at the INDEX roadmap.

Practice sections Every module has a *Practice* — *where to drill this module* table: specific problems by number and title, plus CSES and Codeforces tag pages. **Every LeetCode slug was verified against live search**, not written from memory.

C++ Toolkit per module A primer on the C++ features that module's code leans on, grounded in Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., cited by section and page.

Appendix per module A heavily-commented, runnable C++ translation of **every** pseudocode block, kept line-for-line faithful to the pseudocode (1-based indexing and all).

Links under every pseudocode block **95** → `C++ implementation:` links, covering all **71** procedure-pseudocode blocks plus 24 algorithmic recurrences and formulas (M11's DP recurrences, M07's hash functions, M02's summations).

Per-module coverage

MODULE	PSEUDOCODE BLOCKS	C++ LINKS	APPENDIX CODE BLOCKS
M01 Foundations	6	6	6
M02 Asymptotics	1	6	6
M03 Divide & Conquer	3	5	5
M04 Randomization	5	5	5
M05 Sorting	10	10	11
M06 Elementary Data Structures	3	3	3
M07 Hashing	3	9	7
M08 Search Trees	17	17	18
M09 Amortized Analysis	3	3	3
M10 Union-Find	5	5	7
M11 Dynamic Programming	0 (recurrences)	11	11
M12 Greedy	2	2	2
M13 Graph Traversal	4	4	5
M14 Minimum Spanning Trees	3	3	4
M15 Shortest Paths	6	6	7

Body code vs appendix code

They are deliberately different, and comparing them is the point:

- **Body implementations** are the *practical* version — what you would actually write, 0-indexed, using the STL where it helps.
- **Appendix implementations** are *literal translations of the pseudocode*, line for line, keeping CLRS's 1-based indexing and variable names, with comments explaining both the algorithm and the C++ decision behind each line.

How to use these notes

1. Read a module's **Big Idea** and **What You Should Be Able To Do**.
2. Work through the body, following each [→ C++ implementation:](#) link when you

want to see the pseudocode become code.

3. Read the **C++ Toolkit** once per module — it explains the language features that module's code depends on.
 4. Drill the **Practice** table.
 5. Revise from the **One-Page Recall / Chapter in One Page** section at the end, not from the whole module.
-

Sources

BOOK	USED FOR
Steven S. Skiena, <i>The Algorithm Design Manual</i> , 3rd ed.	intuition, modelling, pattern recognition, war stories, implementation pragmatics
Cormen, Leiserson, Rivest, Stein, <i>Introduction to Algorithms</i> , 4th ed.	formal definitions, invariants, proofs, recurrences, rigorous complexity
Mark Allen Weiss, <i>Data Structures and Algorithm Analysis in C++</i> , 4th ed.	the C++ language teaching only — cited as [Weiss §1.5.3, p.25]

Citations look like [CLRS §2.1, p.17] and [Skiena §1.3, p.11] so you can go back to the source quickly.

Still to be written

M16–M27 (network flow, backtracking, strings, NP-completeness, approximation, number theory, linear programming, FFT, parallel/online, ML, geometry, and the master cheat sheet). The INDEX carries the full roadmap with status markers.